

Relatório – Pipeline de Limpeza e Preparação de Texto para SLMs

Licenciatura em Engenharia Informática – 1º Ano

Unidade Curricular: Laboratório de Programação

Ano Letivo: 2025/2026

Elementos do Grupo:

- Afonso Pereira – 2025167975
- Francisco Capitão – 2025168823
- Tomás Monteiro – 2025156384

1. Introdução

No âmbito da unidade curricular de Laboratório de Programação, foi desenvolvido um sistema de normalização textual baseado numa pipeline de pré-processamento e integração com Small Language Models (SLMs). A aplicação permite carregar documentos em diferentes formatos, extrair e limpar texto automaticamente e preparar os dados para modelos de linguagem.

A necessidade de pré-processamento textual surge devido à presença de ruído em documentos digitais, como PDFs digitalizados, erros de encoding, caracteres inválidos e problemas de formatação. O sistema desenvolvido automatiza este processo, garantindo uma melhoria significativa da qualidade do texto.

2. Objetivos do Projeto

Os principais objetivos definidos para este projeto foram:

- Implementar upload de ficheiros em múltiplos formatos;
- Extrair texto bruto preservando inconsistências;
- Criar uma pipeline configurável de limpeza textual;
- Dividir automaticamente o texto em chunks;
- Preparar prompts para modelos SLM;
- Integrar comunicação com APIs de modelos de linguagem;
- Gerar relatórios automáticos dos resultados obtidos;
- Avaliar o impacto da normalização textual.

3. Tecnologias Utilizadas

Durante o desenvolvimento do projeto foram utilizadas várias bibliotecas e tecnologias:

- Python 3 – linguagem principal;
- Streamlit – interface gráfica;

- pypdf / pdfplumber – extração de PDFs;
- python-docx – leitura de DOCX;
- pytesseract e pdf2image – OCR para PDFs digitalizados;
- requests – comunicação HTTP com a API;
- langdetect – detecção automática de idioma;
- regex (re) – limpeza textual;
- pyspellchecker – detecção ortográfica;
- Git/GitHub – controlo de versões.

4. Arquitetura da Aplicação

A aplicação foi desenvolvida com uma arquitetura modular, dividida em vários componentes:

1. módulo de Upload – responsável pelo carregamento de ficheiros.
2. módulo de extração – leitura de conteúdo textual em PDF, DOCX e TXT.
3. Pipeline de Pré-processamento – limpeza e normalização textual.
4. módulo de Chunking – divisão do texto em blocos.
5. comunicação com API – envio de prompts para o SLM.
6. Geração de Relatórios – exportação automática em HTML ou PDF.

5. Implementação do Sistema

O sistema foi dividido em diferentes módulos Python para tornar o código mais organizado e modular.

Principais ficheiros:

- app.py – interface principal da aplicação;
- file_readers.py – extração de texto;
- text_pipeline.py – limpeza e preparação textual;
- api_handler.py – comunicação com o modelo;
- report_generator.py – geração de relatórios.

A aplicação suporta TXT, DOCX e PDF. Em PDFs digitalizados é utilizado OCR através do Tesseract.

6. Pipeline de Limpeza e Pré-processamento

Após a extração, o texto passa por várias etapas de processamento:

- Remoção de artefactos;
- normalização Unicode;
- normalização de espaços;

- Correção de quebras de linha;
- Reconstrução de parágrafos;
- Remoção de cabeçalhos e rodapés;
- Detecção ortográfica;
- Detecção automática de idioma.

A interface permite visualizar o texto antes e depois da normalização.

7. Preparação do Input para o SLM

Depois da limpeza, o sistema divide o texto em chunks menores para evitar limites de contexto dos modelos.

Cada chunk gera automaticamente um prompt formatado, por exemplo:

"Normaliza o seguinte texto mantendo o significado original."

8. comunicação com a API

A comunicação com o modelo de linguagem é realizada através de pedidos HTTP POST.

Fluxo de funcionamento:

1. Criação do prompt;
2. Envio para o modelo SLM;
3. Receção da resposta;
4. Apresentação do resultado ao utilizador.

Foi utilizada uma API baseada no modelo llama-3.2-1b-instruct.

9. Interface da Aplicação

A interface foi desenvolvida em Streamlit com foco na simplicidade e usabilidade.

Funcionalidades disponíveis:

- Upload de ficheiros;
- Visualização do texto bruto;
- Comparação antes/depois;
- Visualização de chunks;
- Envio para o modelo;
- Exportação de relatórios.

10. Testes e Resultados

Foram realizados testes com documentos PDF, DOCX e TXT.

Resultados observados:

- Correção de quebras de linha;
- Remoção de caracteres inválidos;
- Melhoria significativa da legibilidade;
- Texto final mais consistente e organizado.

11. Dificuldades Encontradas

Durante o desenvolvimento surgiram vários desafios:

- extração inconsistente de texto em PDFs;
- Problemas de encoding;
- PDFs complexos com tabelas e imagens;
- Limitações do OCR;
- Gestão do contexto dos modelos SLM.

Estas dificuldades foram ultrapassadas através de testes sucessivos e técnicas de limpeza incremental.

12. Melhorias Futuras

Possíveis melhorias futuras incluem:

- Correção ortográfica automática avançada;
- Suporte a mais formatos;
- Utilização de modelos locais;
- Melhoria do OCR;
- Suporte a múltiplos idiomas;
- Exportação avançada em PDF.

13. Conclusão

O projeto desenvolvido cumpre os objetivos propostos, permitindo extração multi-formato, limpeza textual, chunking, geração de prompts e integração com modelos SLM.

Além disso, foram implementadas funcionalidades adicionais como OCR e detecção ortográfica, tornando a aplicação mais robusta e próxima de cenários reais de processamento textual.